

Database Project

Prepared by: Vida Sadati

Instructor: Dr. Soudeh Hosseini

The following tables are used in this project:

Stud (S# , Sname , City , Avg , Clg#)

Crs (C# , Cname , Unit , Clg#)

Prof (Pname , Office , Degree , Esp , Clg#)

Clg (Clg# , Clgname , Pname , City)

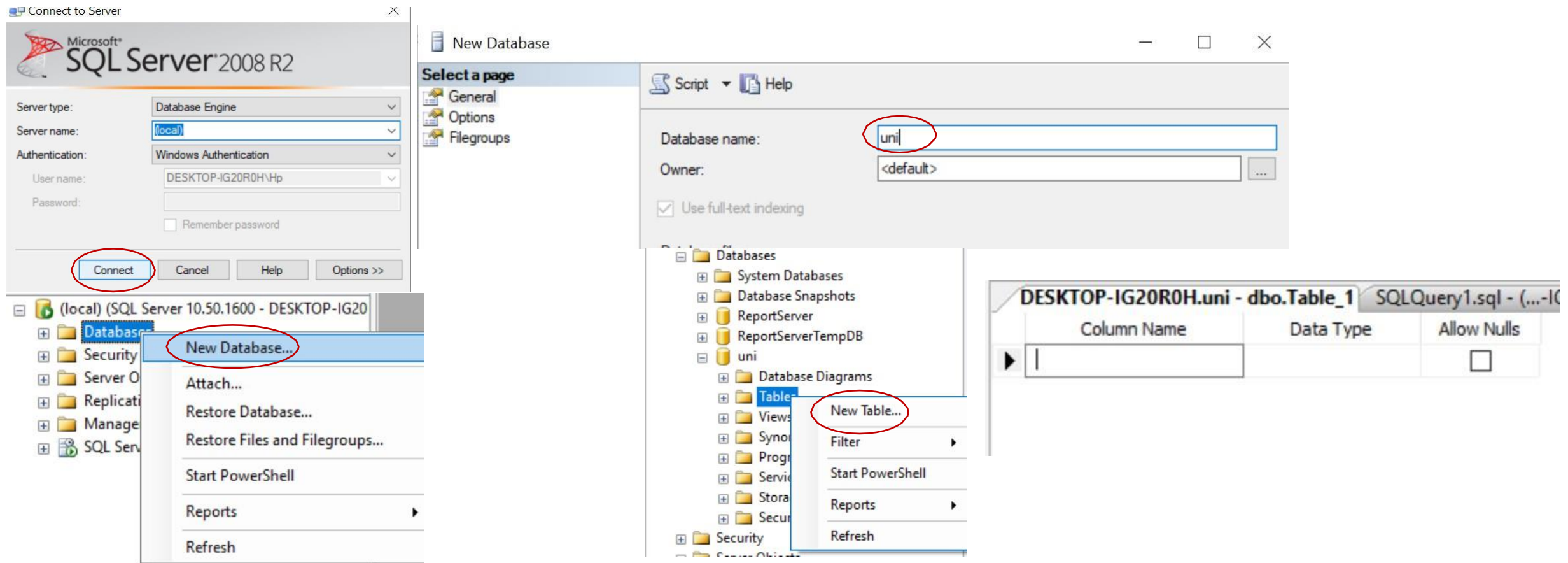
Sec (Sec# , C# , S# , Term , Pname , Score)

This project was developed using **Microsoft SQL Server Management Studio (SSMS)** for database design and **Microsoft Visual Studio** with **C# .NET** for the graphical user interface. The database is named **Uni**.

To create the database:

1. Open Microsoft SQL Server Management Studio.
2. In **Object Explorer**, right-click on **Databases**.
3. Select **New Database**.
4. Enter the name **Uni** and create the database.

After creating the database, the tables must be designed.



To create a table:

1. Right-click on **Tables**.
2. Select **New** → **Table**.
3. Define the column names and their corresponding data types.

Using the **Allow Nulls** option, we can specify whether a field can contain NULL values. Primary keys can never be NULL.

To define a primary key:

1. Right-click the desired field.
2. Select **Set Primary Key**.

For composite primary keys, use the **Shift** key to select multiple fields before setting the primary key.

	Column Name	Data Type	Allow Nulls
	S#	int	☒
	Sname	nvarchar(50)	☒
	City	nvarchar(50)	☒
	Avg	float	☒
▶	Clg#	int	☒
			☐

Column Name	Data Type	Allow Nulls
S#		
Sname		
City		
Avg		
Clg#		

- Set Primary Key
- Insert Column
- Delete Column
- Relationships...
- Indexes/Keys...
- Fulltext Index...
- XML Indexes...

CHECK Constraints

To apply a CHECK constraint:

1. Right-click the desired column.
2. Select **Check Constraints**.
3. Enter the required validation rule.

These constraints ensure that only valid data can be inserted into the database.

The screenshot illustrates the steps to apply a CHECK constraint in SQL Server Enterprise Manager. On the left, a table with columns S#, Sname, City, Avg, and Clg# is shown. The 'Avg' column is selected, and the 'Check Constraints...' option is highlighted in the context menu. The main window, 'Check Constraints', shows a list of constraints with 'CK_Table_1' selected. The 'Expression' property is set to 'Avg >= 0 and Avg <= 20'. A dialog box, 'Check Constraint Expression', is open, showing the same expression in the 'Expression:' field. Red circles highlight the 'Check Constraints...' menu item, the 'CK_Table_1' constraint, the 'Expression' property, and the 'Expression:' field in the dialog box.

Column Name	Data Type	Allow Nulls
S#	int	☐
Sname	nvarchar(50)	☒
City	nvarchar(50)	☒
Avg	float	☒
Clg#		

Check Constraints

Selected Check Constraint: CK_Table_1

Editing properties for existing check constraint.

(General)

Expression: Avg >= 0 and Avg <= 20

(General)

Expression: Avg >= 0 and Avg <= 20

Identity

(Name): CK_Table_1

Description:

Table Designer

Check Existing Data On Create: Yes

Enforce For INSERTs And UPDA: Yes

Enforce For Replication: Yes

Check Constraint Expression

Expression: Avg >= 0 and Avg <= 20

OK Cancel

sec

	Sec#	int	☒
►	C#	int	☐
►	S#	int	☐
►	Term	nvarchar(50)	☐
	Pname	nvarchar(50)	☒
	Score	int	☒
			☐

crs

	Column Name	Data Type	Allow Nulls
►	C#	int	☐
	Cname	nvarchar(50)	☒
	Unit	int	☒
	Clg#	int	☒
►			☐

stud

	Column Name	Data Type	Allow Nulls
►	S#	int	☐
	Sname	nvarchar(50)	☒
	City	nvarchar(50)	☒
	Avg	float	☒
	Clg#	int	☒
			☐

clg

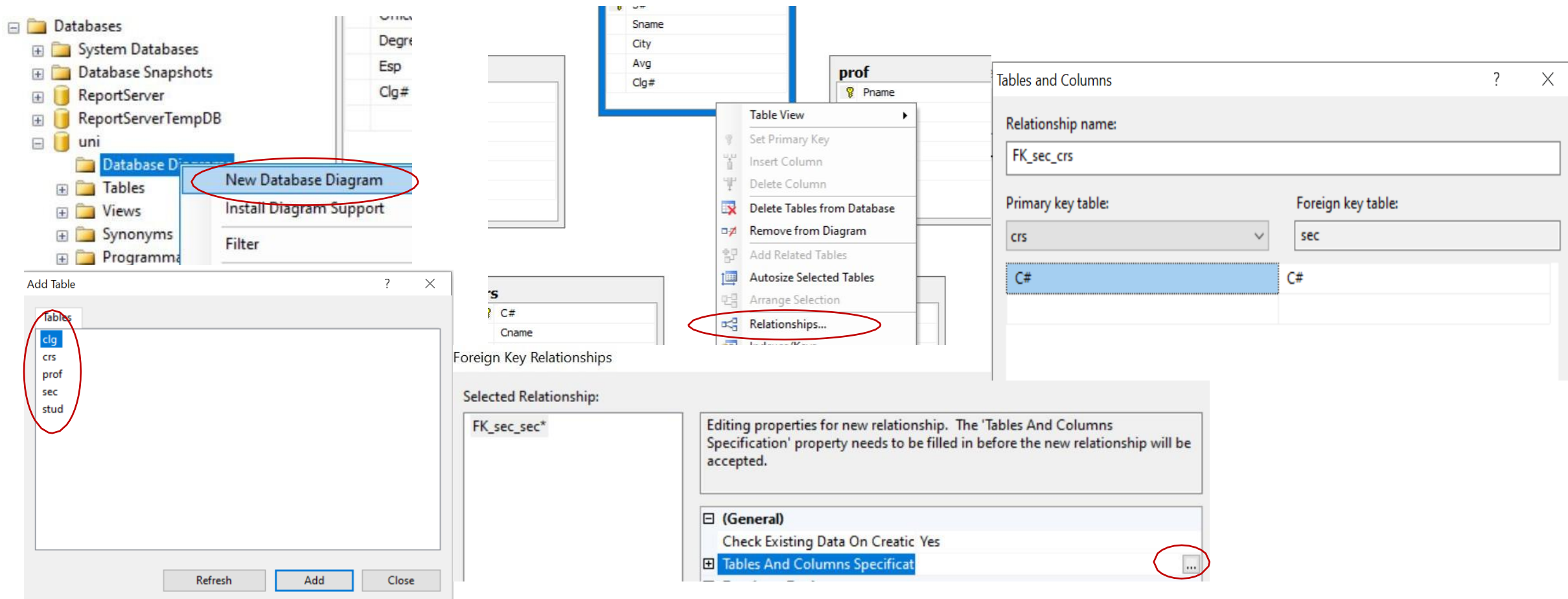
	Column Name	Data Type	Allow Nulls
►	Clg#	int	☐
	Clgname	nvarchar(50)	☒
	Pname	nvarchar(50)	☒
	City	nvarchar(50)	☒
			☐

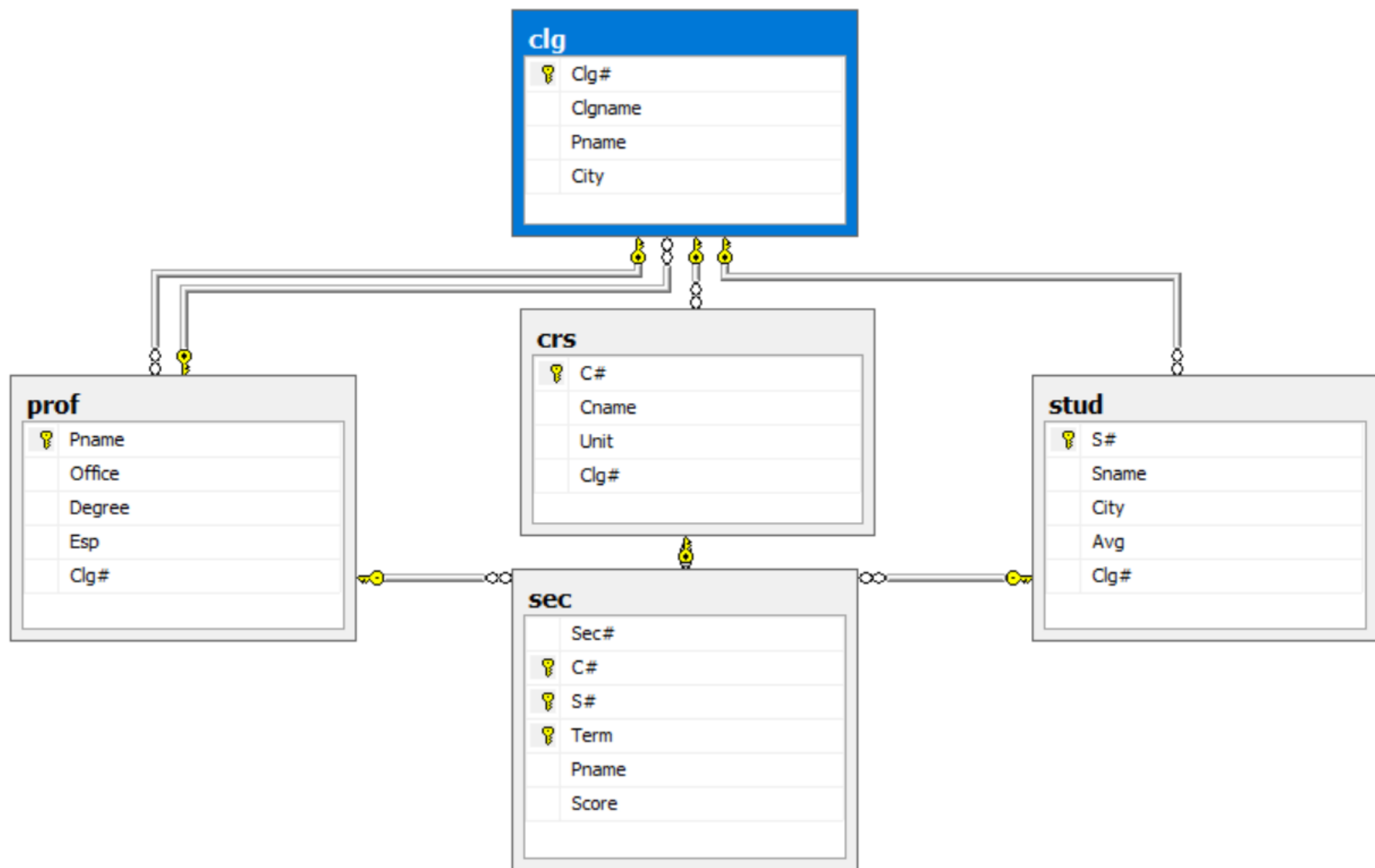
prof

	Column Name	Data Type	Allow Nulls
►	Pname	nvarchar(50)	☐
	Office	nvarchar(50)	☒
	Degree	nvarchar(50)	☒
	Esp	nvarchar(50)	☒
	Clg#	int	☒
			☐

To create relationships:

1. Right-click **Database Diagrams**.
2. Select **New Database Diagram**.
3. Add the required tables.
4. Define relationships between tables using foreign keys.



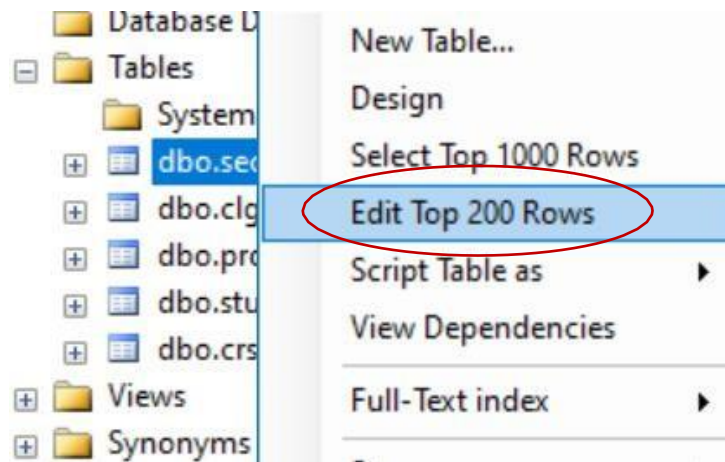


Populating Tables

Before creating relationships, tables should be populated with data.

To insert data manually:

1. Right-click the desired table.
2. Select **Edit Top 200 Rows**.
3. Enter the values according to the table structure.

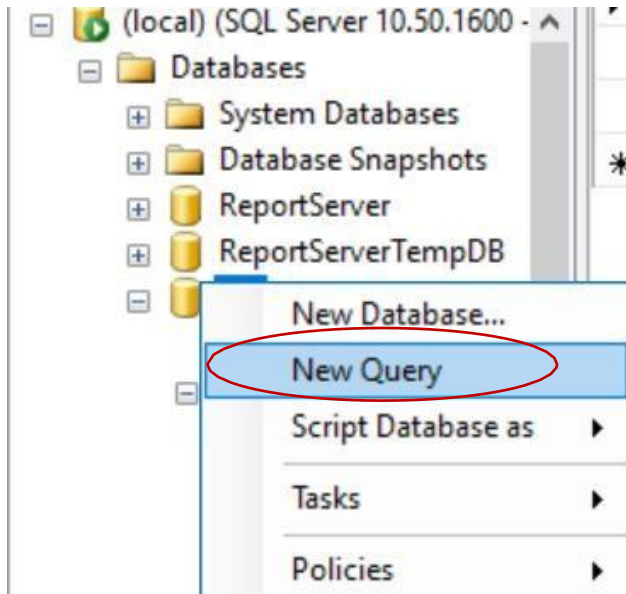


	Pname	Office	Degree	Esp	Clg#
▶	chaboki	w1	professor	NULL	23
	marjan kuchaki	w1	professor	NULL	23
	najmeh mansuri	w1	professor	NULL	23
	niktabE	i5	professor	NULL	10
	rostami	m1	professor	NULL	24
	sudeh hosseini	w1	professor	NULL	23
	tohidi	I5	professor	NULL	10
	zanguyi	m1	professor	NULL	24
*	NULL	NULL	NULL	NULL	NULL

Inserting New Records

To add a new record, the **INSERT** statement is used.

```
INSERT INTO TableName (...)  
VALUES (...)
```



```
insert stud values (400375019, 'darya', 'tehran', 19, 23)
```

Creating Views

Before creating a View, the related tables must contain data.

When inserting records:

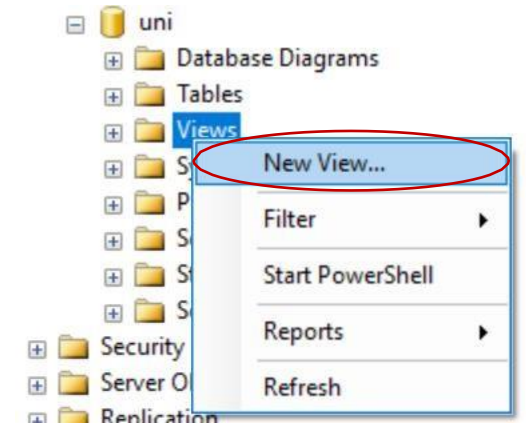
- Pay attention to NULL and NOT NULL constraints.
- Consider foreign key relationships.
- Start by inserting data into tables that do not contain foreign keys.

To create a View:

1. Right-click **Views**.
2. Select **New View**.
3. Add the required tables.
4. Select the desired columns (the primary key should always be included).
5. Write the appropriate query.

Any modification performed through a View is reflected in the underlying table, including updates and deletions.

The Execute button is used to run the View query.



Example 1: High GPA Students

Create a View for students with high grade averages using the **Stud** table.

stud

☐ * (All Columns)

☒ S#

☒ Sname

☐ City

☒ Avg

<

Column	Alias	Table	Output	Sort Type	Sort Order	Filter
S#		stud	☒			
Sname		stud	☒			
Avg		stud	☒			>= 17

SELECT S#, Sname, Avg
FROM dbo.stud
WHERE (Avg >= 17)

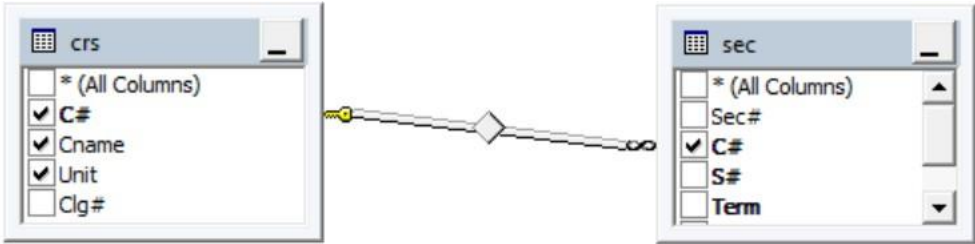
	S#	Sname	Avg
▶	400370045	kiana rostami	17
	400375018	vida sadati	17
	400378945	Darya	19
	478912356	sara	18
*	NULL	NULL	NULL

stud

S#	Sname	City	Avg	Clg#
49787878	ali	NULL	16.5	NULL
400145679	mohamad	alborz	15	23
400370045	kiana rostami	kerman	17	23
400375017	RANDOM	kerman	16	24
400375018	vida sadati	kerman	17	23
400378945	Darya	tehran	19	23
478912356	sara	shiraz	18	24
NULL	NULL	NULL	NULL	NULL

Example 2: Offered Courses

Create a View for offered courses using the **Sec** and **Crs** tables.



Column	Alias	Table	Output	Sort Type	Sort Order	Filter
C#		crs	☒			
Cname		crs	☒			
Unit		crs	☒			
C#	Expr1	sec	☒			

```
CT 1  dbo.crs.C#, dbo.crs.Cname, dbo.crs.Unit, dbo.sec.C# AS Expr1
      dbo.crs INNER JOIN
      dbo.sec ON dbo.crs.C# = dbo.sec.C#
```

C#	Cname	Unit	Expr1
123450	math 2	3	123450
123454	differential	3	123454
123454	differential	3	123454
123455	andishe1	2	123455
123455	andishe1	2	123455
123456	database	4	123456
123456	database	4	123456

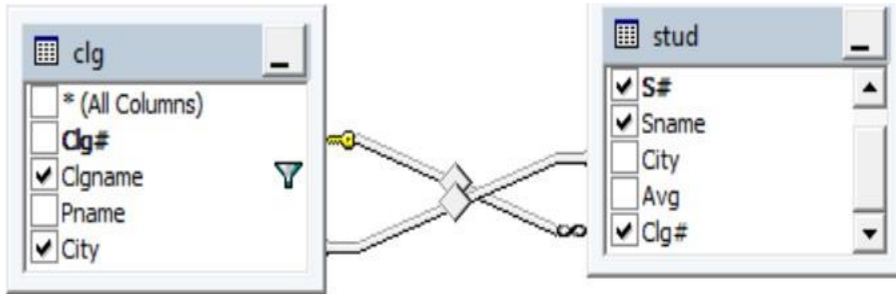
	C#	Cname	Unit	Clg#
▶	123450	math 2	3	24
	123451	emamat	2	10
	123454	differential	3	24
	123455	andishe1	2	10
	123456	database	4	23
	123457	operating system	4	23
	123458	data structure	4	23
	123459	principle of comp...	4	23
*	NULL	NULL	NULL	NULL

sec

	Sec#	C#	S#	Term	Pname	Score
▶	4	123450	400375018	1400-2	rostami	20
	6	123454	400370045	1401-2	zanguyi	19
	3	123454	400375018	1401-2	zanguyi	19
	5	123455	400375018	1401-2	niktabE	19
	5	123455	400378945	1401-1	niktabE	18
	1	123456	400375017	1401-2	sudeh hosseini	18
	1	123456	400375018	1401-2	sudeh hosseini	20
	2	123457	400370045	1401-2	marjan kuchaki	19
*	NULL	NULL	NULL	NULL	NULL	NULL

Example 3: Native Students of the Computer Science College

Create a View using the **Stud** and **Clg** tables.



	Clg #	Clgname	Pname	City
	10	theology	tohidi	kerman
	23	computer science	najmeh mansuri	kerman
	24	math	rostami	kerman
►*	NULL	NULL	NULL	NULL

	Column	Alias	Table	Output	Sort Type	Sort Order	Filter
►	City		clg	☒			
	S#		stud	☒			
	Sname		stud	☒			
	Clgname		clg	☒			= 'computer ...
	Clg#		stud	☒			

```

SELECT dbo.clg.City, dbo.stud.S#, dbo.stud.Sname, dbo.clg.Clgname, dbo.stud.Clg#
FROM   dbo.clg INNER JOIN
       dbo.stud ON dbo.clg.Clg# = dbo.stud.Clg# AND dbo.clg.City = dbo.stud.City
WHERE  (dbo.clg.Clgname = 'computer science')
    
```

stud

	S#	Sname	City	Avg	Clg#
	49787878	ali	NULL	16.5	NULL
	400145679	mohamad	alborz	15	23
	400370045	kiana rostami	kerman	17	23
	400375017	RANDOM	kerman	16	24
	400375018	vida sadati	kerman	17	23
	400378945	Darya	tehran	19	23
	478912356	sara	shiraz	18	24
►*	NULL	NULL	NULL	NULL	NULL

For testing purposes, data may be entered manually; however, this approach is suitable only for simple cases.

stud

☐ * (All Columns)

☒ S#

☒ Sname

☐ City

☒ Avg

SELECT

S#, Sname, Avg

FROM

dbo.stud

WHERE

(Avg >= 17)

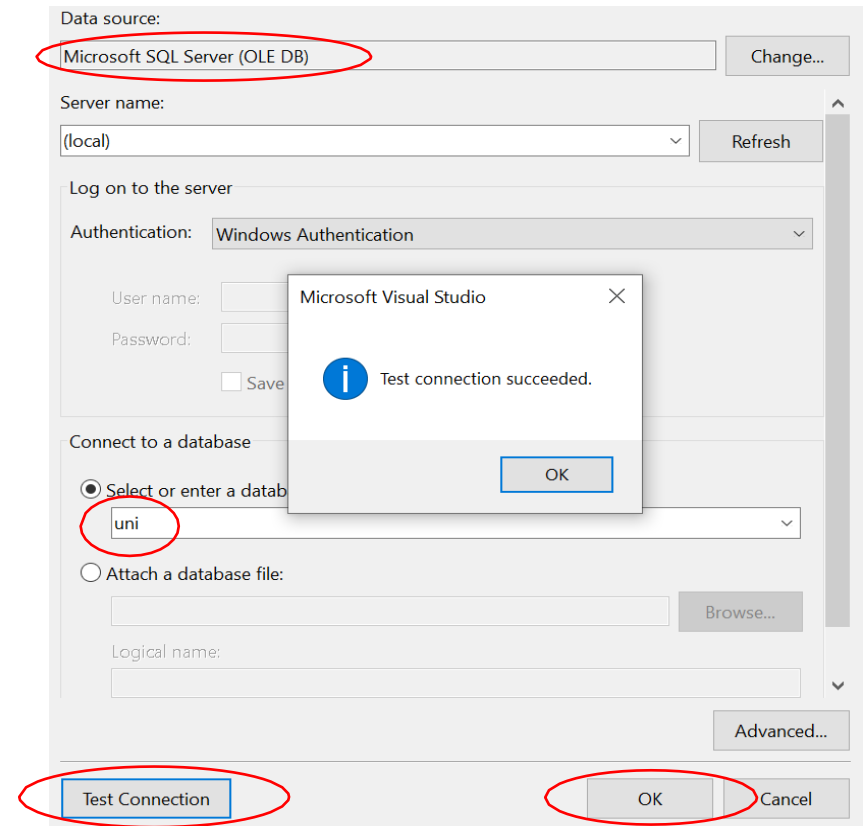
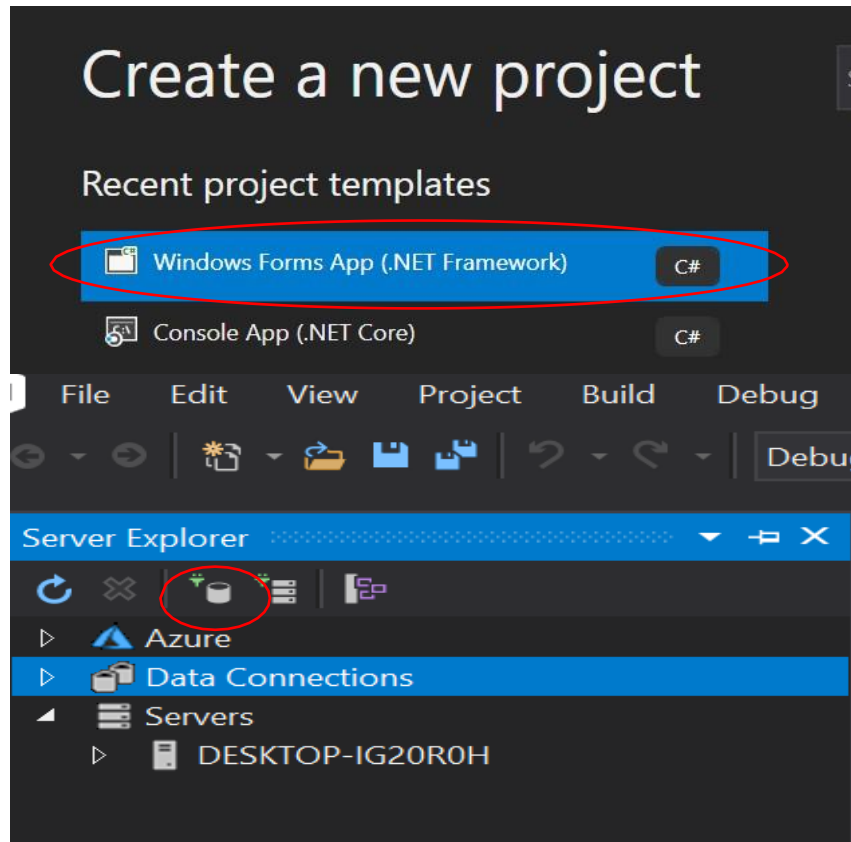
	S#	Sname	City	Avg	Clg#
	49787878	ali	NULL	16.5	NULL
	400145679	mohamad	alborz	15	23
	400370045	kiana rostami	kerman	17	23
	400375017	RANDOM	kerman	16	24
	400375018	vida sadati	kerman	17	23
▶	400378145	mahla	NULL	17	NULL
	400378945	Darya	tehran	19	23
	478912356	sara	shiraz	18	24
*	NULL	NULL	NULL	NULL	NULL

Graphical User Interface (Microsoft Visual Studio)

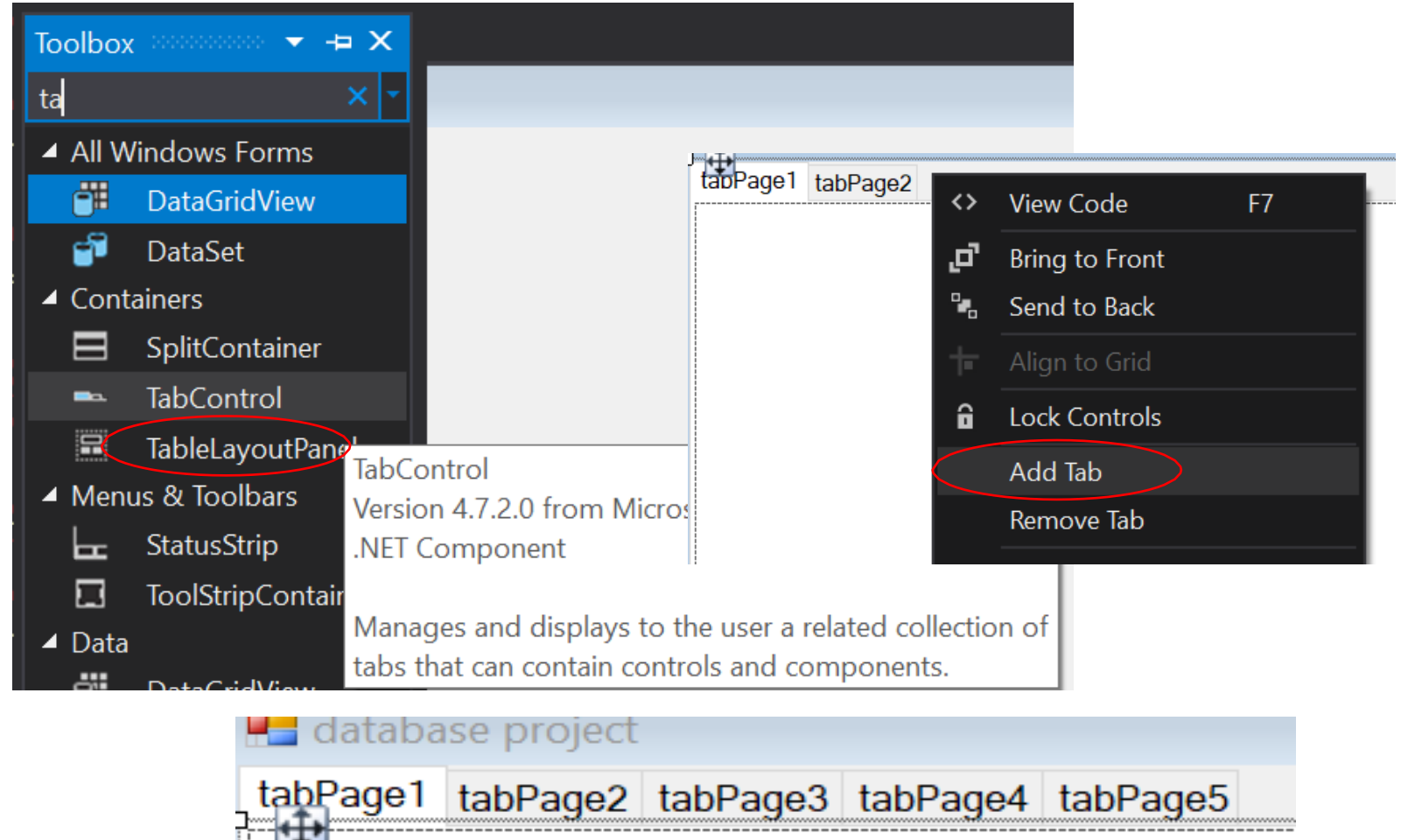
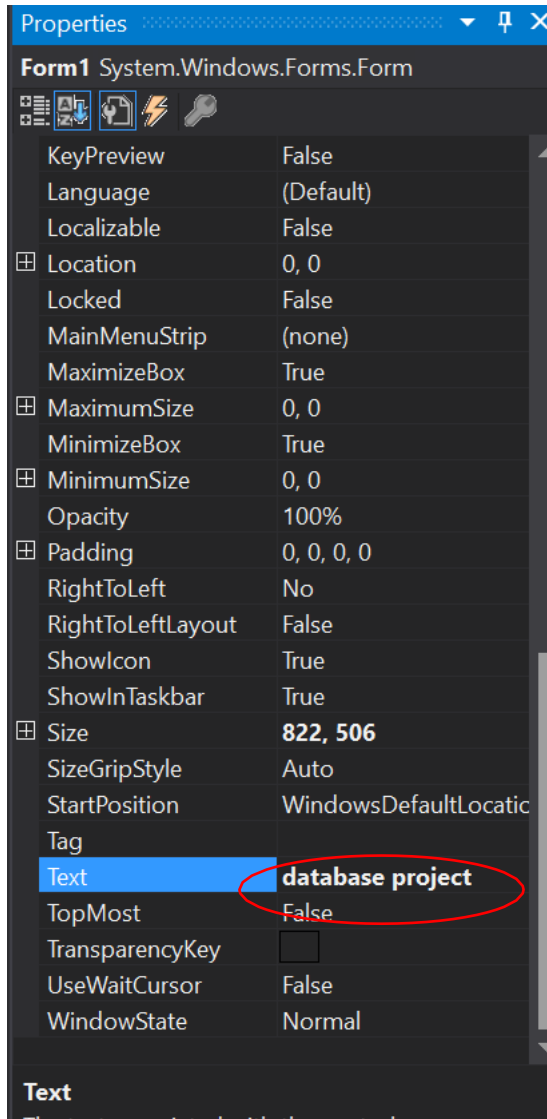
To create the graphical interface:

1. Open Microsoft Visual Studio.
2. Create a **Windows Forms App (.NET Framework)** project.
3. Assign a project name.
4. Open the design environment.

From **Server Explorer**, create a connection to the database.



The project interface uses **TabControl** components.
Additional tabs can be added as needed to organize the application into multiple sections.

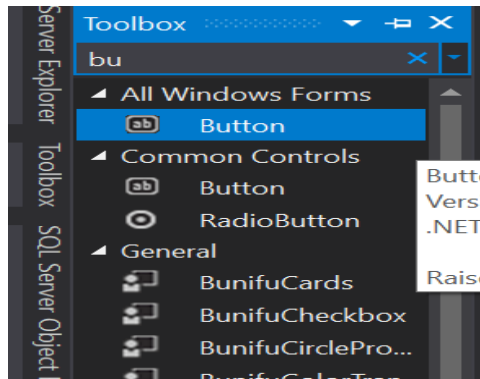


Interface Components

The following controls are used throughout the application:

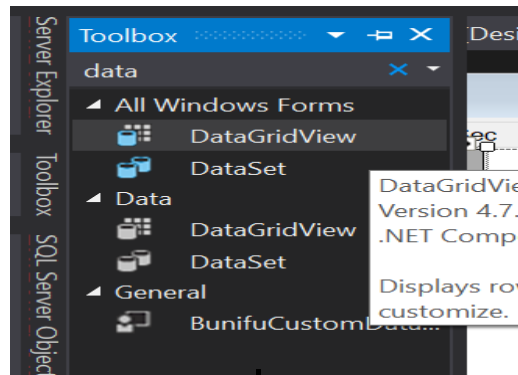
- Label (for captions and titles)
- TextBox (for user input)
- Button (for executing commands)
- DataGridView (for displaying database records)
- TabControl (for organizing sections)

Each table in the database has its own dedicated interface for data management.

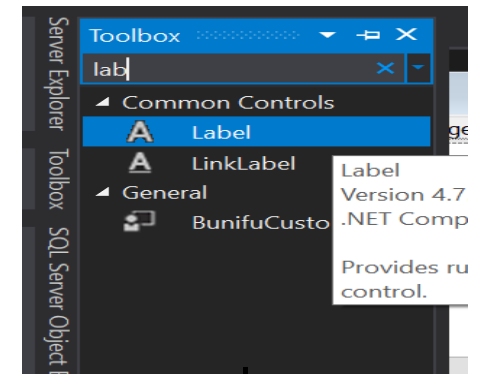
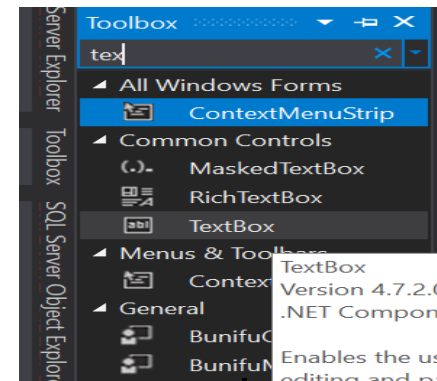


Save Delete

Refresh table



S#	Sname	City	Avg	Clg#
49787878	ali		16.5	
400375017	RANDOM	kerman	16	23
400375018	vida sadati	kerman	17	23
400378945	Darya	tehran	19	23



STUDENT number(S#):

STUDENT name(Sname):

City:

Average(Avg):

College number(Clg#):

Stud Crs Prof Clg Sec

Student Table

Save

Delete

disired word

Edit

Refresh table

Student number(S#):

Student name(Sname):

City:

Average(Avg):

College number(Clg#):

	S#	Sname	City	Avg	Clg#
▶	400145679	mohamad	alborz	15	24
	400370045	kiana rostami	kerman	17	23
	400375017	RANDOM	kerman	16	24
	400375018	vida sadati	kerman	17	23
	400375070	RANDOM2	kerman	16	22

Stud	Crs	Prof	Clg	Sec
------	-----	------	-----	-----

Course Table

Save

Delete

disired word

Edit

Refresh table

Course number(C#):

Course name(Cname):

Unit:

College number(Clg#):

	C#	Cname	Unit	Clg#
▶	123450	math 2	3	24
	123451	emamat1	2	10
	123454	differential	3	24
	123455	andishe1	2	10
	123456	database	4	23

Stud Crs Prof Clg Sec

Professor Table

Save

Delete

disired word

Edit

Refresh table

Professor name(Pname):

Office:

Degree:

Specialize(ESP):

College number(Clg#):

	Pname	Office	Degree	Esp	Clg#
▶	chaboki	w1	professor		23
	marjan kuchaki	w1	professor1		23
	najmeh mansuri	w1	professor		23
	niktabE	i5	professor		10
	rostami	m1	professor		24

Stud Crs Prof Clg Sec

College Table

Save

Delete

disired word

Edit

Refresh table

College number(Clg#):

College name(Clgname):

Professor name(Pname):

City:

	Clg#	Clgname	Pname	City
▶	10	theology	tohidi	kerman
	23	computer science	najmeh mansuri	kerman
	24	math	rostami	kerman
*				

Section Table

Save

Delete

disired word

Edit

Refresh table

Section number(Sec#):

Course number(Crs#):

Student number(S#):

Term:

Professor name(Pname):

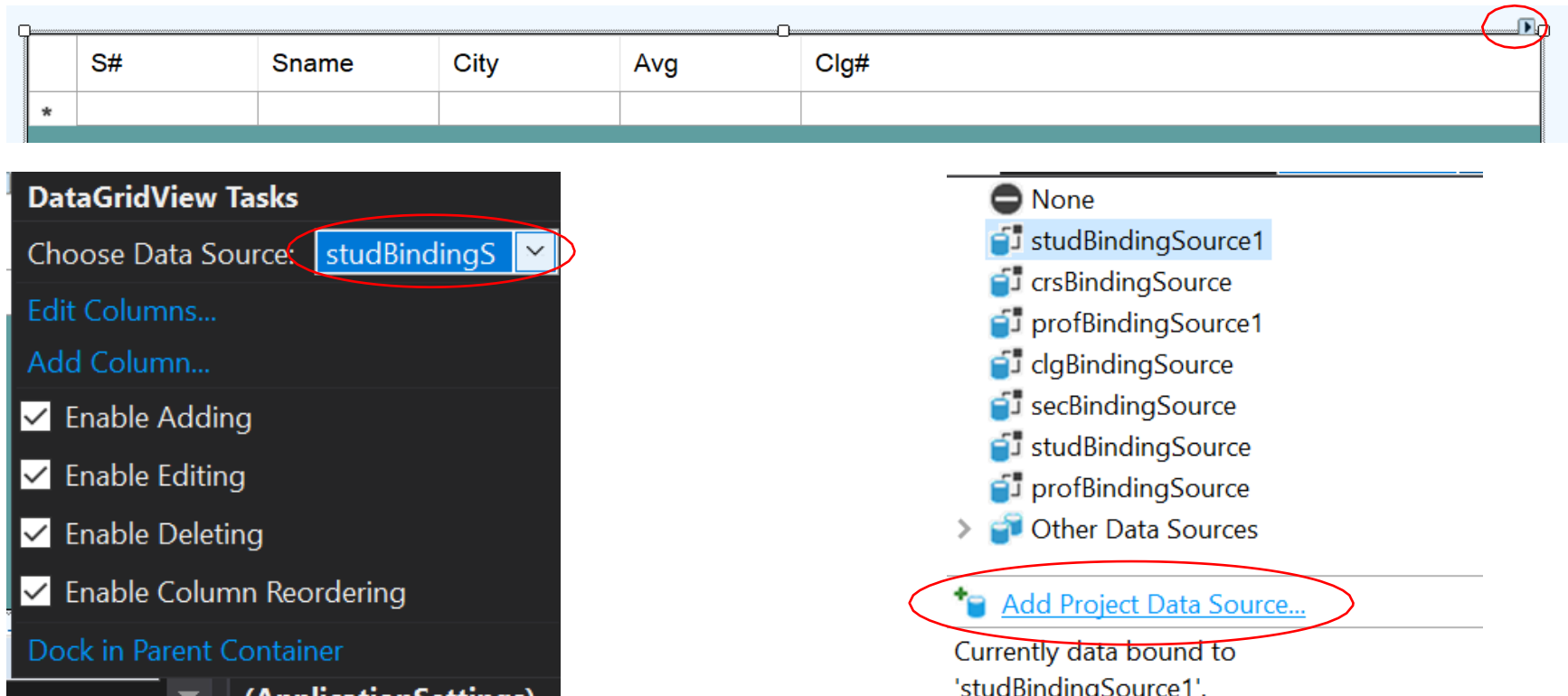
Score:

	Sec#	C#	S#	Term	Pname	Score
►	4	123450	400375018	1400-2	rostami	20
	6	123454	400370045	1401-2	zanguyi	19
	3	123454	400375018	1401-2	zanguyi	19
	5	123455	400375018	1401-2	niktabE	19

DataGridView Configuration

To configure a DataGridView:

1. Drag the control onto the form.
2. Resize it appropriately.
3. Configure its properties according to project requirements.
4. Connect it to the database source.



The image shows a DataGridView control on a form and its configuration in the Visual Studio Properties window. The DataGridView has columns: S#, Sname, City, Avg, and Clg#. The Properties window shows the 'DataGridTasks' section with 'Choose Data Source' set to 'studBindingS'. The 'DataSource' property is set to 'studBindingSource1'. The 'Add Project Data Source...' button is highlighted. The status bar indicates 'Currently data bound to 'studBindingSource1'.'

	S#	Sname	City	Avg	Clg#
*					

DataGridView Tasks

Choose Data Source: **studBindingS** ▼

[Edit Columns...](#)

[Add Column...](#)

☒ Enable Adding

☒ Enable Editing

☒ Enable Deleting

☒ Enable Column Reordering

[Dock in Parent Container](#)

[Add Project Data Source...](#)

Currently data bound to 'studBindingSource1'.

 Choose a Data Source Type

Where will the application get data from?


Database


Service


Object

Lets you connect to a database and choose the database objects for your application.

 Choose a Database Model

What type of database model do you want to use?


Dataset

The database model you choose determines the types of data objects your application code uses. A dataset file will be added to your project.

**Choose Your Data Connection**

Which data connection should your application use to connect to the database?

uniConnectionString (Settings) ▼

New C...

This connection string appears to contain sensitive data (for example, a password), which is required to connect to the database. However, storing sensitive data in the connection string can be a security risk. Do you want to include sensitive data in the connection string?

- ☐ No, exclude sensitive data from the connection string. I will set this information in my application code.
- ☐ Yes, include sensitive data in the connection string.

☐ Show the connection string that you will save in the application

< Previous

Next >

Finish

**Choose Your Database Objects**

Which database objects do you want in your dataset?

☑ Tables (partially selected)

- ▷ clg
- ▷ crs
- ▷ prof
- ▷ sec
- ▷ stud
- ▷ Views
- Stored Procedures
- Functions

DataSet name:

kianaDataSet

< Previous

Next >

Finish

Cancel

Final Interface

Refresh table

College number(Clg#):

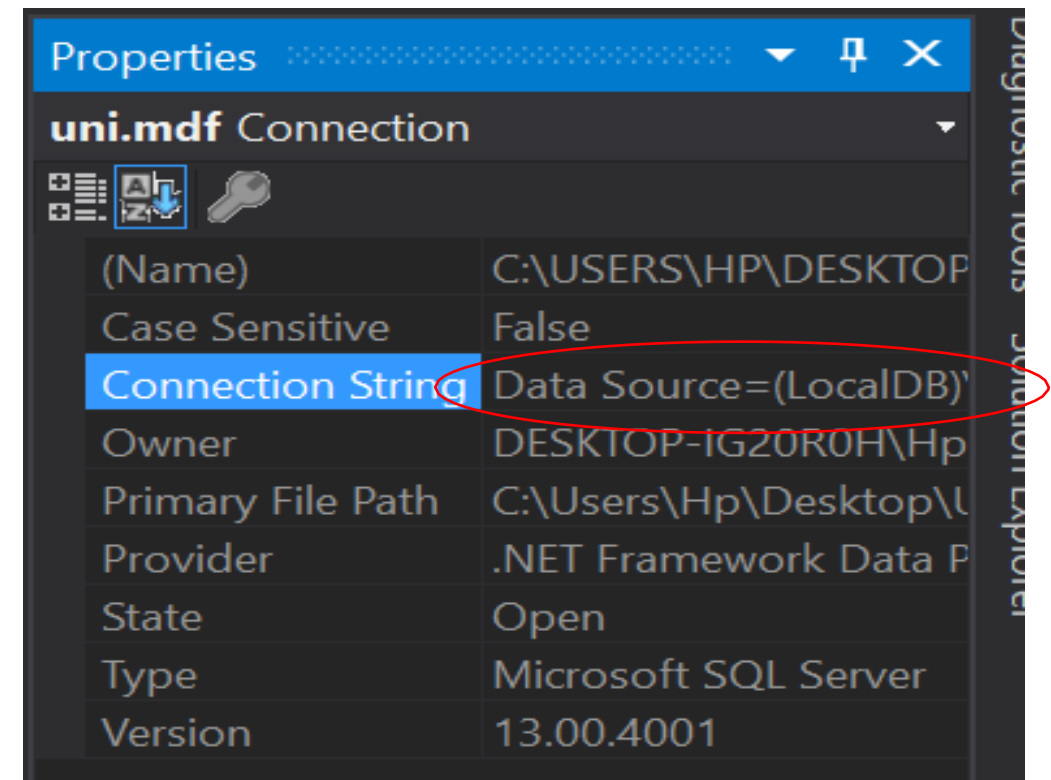
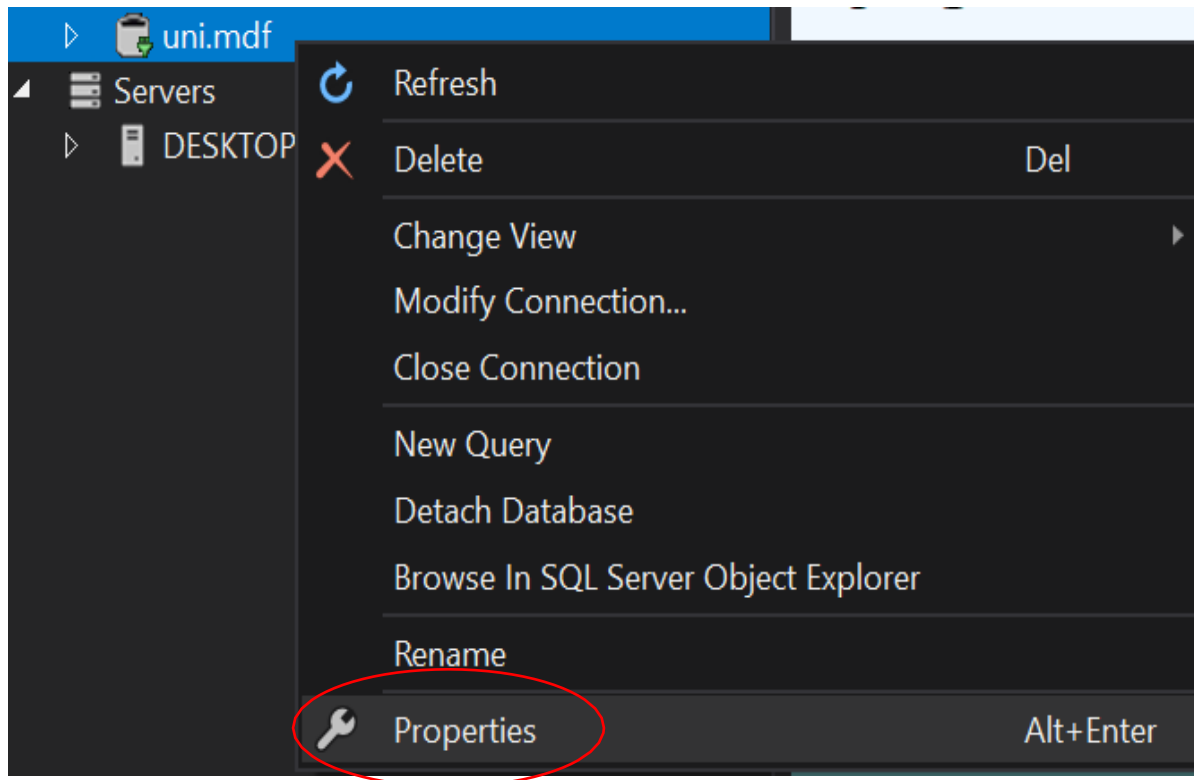
	S#	Sname	City	Avg	Clg#
*					

Database Connection String

Before writing the application code:

1. Right-click the database connection.
2. Select **Properties**.
3. Locate the **Connection String** property.

The Connection String is required for database connectivity within the application.



The necessary namespace should be included at the beginning of the code to prevent compilation errors.

```
1  using System;  
2      using System.Collections.Generic;  
3      using System.ComponentModel;  
4      using System.Data;  
5      using System.Drawing;  
6      using System.Linq;  
7      using System.Text;  
8      using System.Threading.Tasks;  
9      using System.Windows.Forms;  
10     using System.Data.SqlClient;  
11
```

Save

```
1 reference
private void INSERTstud_Click_1(object sender, EventArgs e)
{
    //save stud
    try
    {
        SqlConnection cmd = new SqlConnection();
        cmd.ConnectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Hp\Desktop\Uni project\Uni projec
        SqlCommand mycommand = new SqlCommand();
        mycommand.Connection = cmd;
        mycommand.CommandText = "insert into Stud(S#,Sname,City,Avg,Clg#) values (@S#,@Sname,@City,@Avg,@Clg#)";
        mycommand.Parameters.AddWithValue("@S#", textBox1.Text);
        mycommand.Parameters.AddWithValue("@Sname", textBox2.Text);
        mycommand.Parameters.AddWithValue("@City", textBox3.Text);
        mycommand.Parameters.AddWithValue("@Avg", Convert.ToDouble(textBox4.Text));
        mycommand.Parameters.AddWithValue("@Clg#", textBox5.Text);

        cmd.Open();
        mycommand.ExecuteNonQuery();
        cmd.Close();
        MessageBox.Show("Saved Successfully!");
    }
    catch (Exception e1)
    {
        MessageBox.Show(e1.Message);
    }
}
```


Delete

```
1 reference
private void DELETEstud_Click(object sender, EventArgs e)
{
    //delete stud
    try
    {
        SqlConnection con = new SqlConnection();
        con.ConnectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Hp\Desktop\Uni project\Uni projec
        SqlCommand cmd = new SqlCommand();
        cmd.Connection = con;
        cmd.CommandText = "Delete from stud where S#=@S#";
        cmd.Parameters.AddWithValue("@S#", dataGridView1.CurrentRow.Cells["sDataGridViewTextBoxColumn"].Value);
        con.Open();
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("delete successfully!");
    }
    catch (Exception e1)
    {
        MessageBox.Show(e1.Message);
    }
}
```

Update

1 reference

```
private void Button6_Click(object sender, EventArgs e)
{
    //Edit crs
    try
    {
        SqlConnection con = new SqlConnection();
        con.ConnectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Hp\Desktop\Uni project\Uni pro
        SqlCommand cmd = new SqlCommand();
        cmd.Connection = con;
        cmd.CommandText = "update Crs Set C#=@C#, Cname=@Cname , Unit=@Unit , Clg#=@Clg# Where C#=@C#";

        cmd.Parameters.AddWithValue("@C#", textBox12.Text);
        cmd.Parameters.AddWithValue("@Cname", textBox11.Text);
        cmd.Parameters.AddWithValue("@Unit", textBox10.Text);
        cmd.Parameters.AddWithValue("@Clg#", textBox9.Text);
        con.Open();
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Edited Successfully!");
    }

    catch (Exception e1)
    {
        MessageBox.Show(e1.Message);
    }
}
```

Refresh

1 reference

```
private void Button5_Click(object sender, EventArgs e)
{
    //refresh crs
    try
    {
        SqlConnection con = new SqlConnection();
        con.ConnectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Hp\Desktop\Uni project\Uni
        SqlCommand cmd = new SqlCommand();
        cmd.Connection = con;
        cmd.CommandText = "Select * from Crs";
        DataTable ved = new DataTable();
        SqlDataAdapter ned = new SqlDataAdapter();
        ned.SelectCommand = cmd;
        ned.Fill(ved);
        dataGridView2.DataSource = ved;
    }
    catch (Exception e1)
    {
        MessageBox.Show(e1.Message);
    }
}
```

Design for row in DataGridView

1 reference

```
private void DataGridView1_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    //for select a row in data grid view
    textBox1.Text = dataGridView1.CurrentRow.Cells[0].Value.ToString();
    textBox2.Text = dataGridView1.CurrentRow.Cells[1].Value.ToString();
    textBox3.Text = dataGridView1.CurrentRow.Cells[2].Value.ToString();
    textBox4.Text = dataGridView1.CurrentRow.Cells[3].Value.ToString();
    textBox5.Text = dataGridView1.CurrentRow.Cells[4].Value.ToString();
}
```